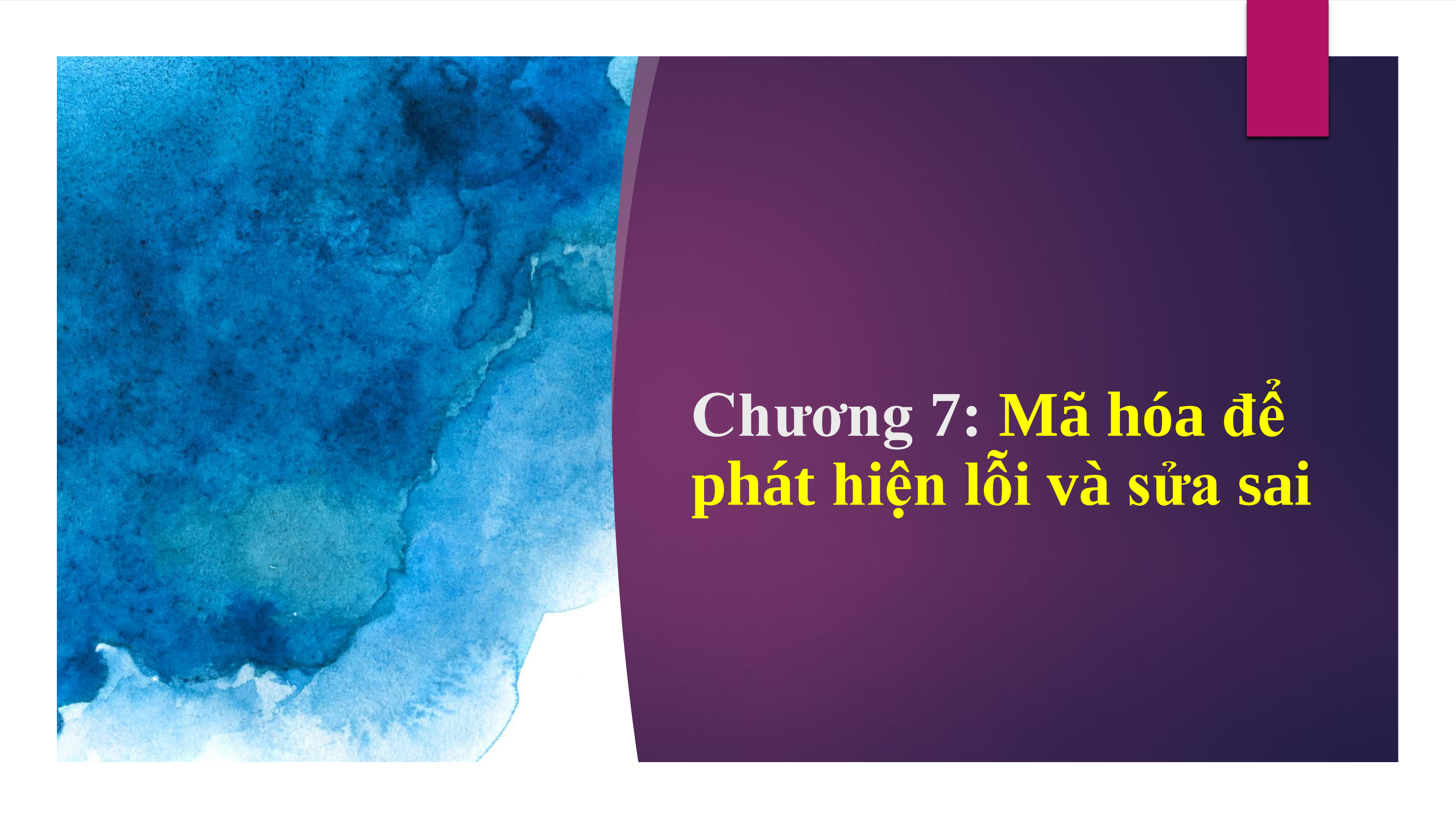




Chương 7: Mã hóa để phát hiện lỗi và sửa sai

Giới thiệu sơ lược

- ▶ Việc sử dụng rộng rãi các kỹ thuật mã hóa để giảm thiểu ảnh hưởng của fading trong truyền dẫn thông tin.
- ▶ “Mã hóa” là việc thêm vào có chủ đích các bit bổ sung trong luồng thông tin kỹ thuật số để cho phép phát hiện và/hoặc sửa lỗi các bit trong luồng thông tin nhận được.
- ▶ Việc truyền dữ liệu chuyển mạch gói trong các hệ thống di động thế hệ thứ ba (3G), cũng như trên các mạng cục bộ không dây, cho thấy các kỹ thuật mã hóa đã và đang được sử dụng phổ biến.

Giới thiệu sơ lược

- ▶ Ba kỹ thuật đã được áp dụng cho các hệ thống không dây:
 - Mã khối để phát hiện lỗi, với các bit phát hiện lỗi được thêm vào mỗi thông điệp được truyền đi.
 - Mã chập để sửa lỗi.
 - Mã Turbo thay cho quy trình mã hóa tích chập trong các hệ thống 3G tốc độ cao. (Mã hóa Turbo là một kỹ thuật mã hóa tương đối gần đây và đặc biệt hiệu quả)

Mã khối

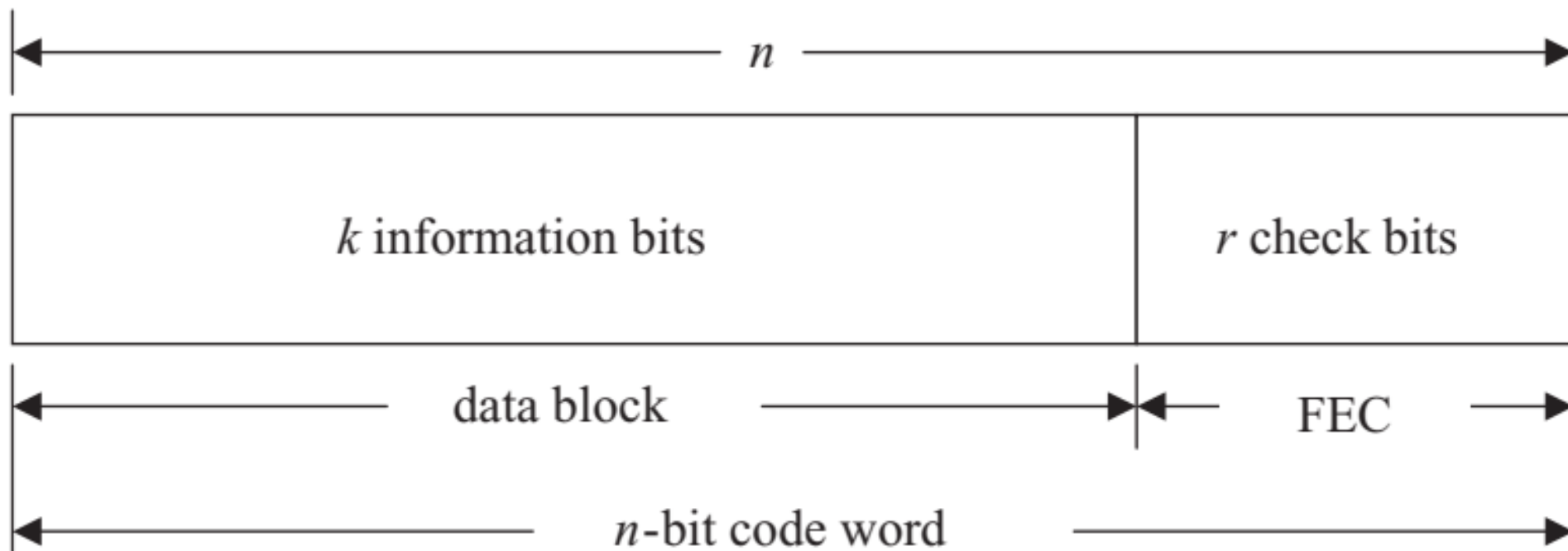


Figure 7.1 Block coding

Mã khối

- ▶ k bit mang thông tin.
- ▶ r bit thêm vào được tạo thành từ các phép biến đổi tuyến tính từ k bit thông tin gọi là các bit kiểm tra.
- ▶ $n = k + r$ là số bit truyền đi tương ứng của 1 từ mã (codeword).
- ▶ Mã được gọi là mã (n, k)

Mã khối

- ▶ Nhiều mã khác nhau đã được phát triển dành cho các mục đích khác nhau:
 - Để phát hiện hoặc sửa lỗi xảy ra độc lập
 - Để phát hiện hoặc sửa lỗi chùm
 - Để cung cấp đồng bộ hóa thông tin nhị phân không có lỗi hoặc ít lỗi.
 - ...

Mã khối

- ▶ Để phát triển các mã có thuộc tính cụ thể, một số lượng đáng kể cấu trúc phải được kết hợp trong chúng.
- ▶ Chúng tôi tập trung vào mã có tính hệ thống, trong đó các bit kiểm tra luôn xuất hiện ở cuối từ mã.
- ▶ Chúng ta sẽ nhấn mạnh đến các mã vòng, mã có thể mô tả được ở dạng đa thức, một đặc tính giúp cho việc triển khai và phân tích chúng trở nên đơn giản hơn nhiều.

Mã khối

- ▶ Vectơ thông tin $d = (d_1, d_2, \dots, d_k)$ với $d_i = 0$ hoặc 1
- ▶ Từ mã được tạo ra $c = (c_1, c_2, \dots, c_k, c_{k+1}, \dots, c_n)$
- ▶ Mã có tính hệ thống tương ứng $d_i = c_i$ với $i = 1 \dots k$
- ▶ r bit kiểm tra từ c_{k+1} đến c_n được tạo thành bằng cách:

$$c_{k+1} = h_{11}d_1 \oplus h_{12}d_2 \oplus \dots \oplus h_{1k}d_k$$

$$c_{k+2} = h_{21}d_1 \oplus h_{22}d_2 \oplus \dots \oplus h_{2k}d_k$$

...

$$c_n = h_{r1}d_1 \oplus h_{r2}d_2 \oplus \dots \oplus h_{rk}d_k$$

Table 7.1 *Codewords, (7,3) code*

d	c
000	000 0000
001	001 1101
010	010 0111
011	011 1010
100	100 1110
101	101 0011
110	110 1001
111	111 0100

Mã khối

Mã khối

- ▶ Với đặc tính hệ thống như trên ta có:

$$c = d.G$$

- ▶ Với G là ma trận $k \times n$ được gọi là ma trận sinh (**code-generator matrix**) với k cột đầu là ma trận đơn vị I_k với kích thước $k \times k$; phần còn lại của ma trận tương ứng với các hệ số h_{ij} của các phương trình ở trên.

Mã khối

► Như vậy:

$$G = [I_k \quad P]$$

► Với:

$$P = \begin{bmatrix} h_{11} & h_{12} & \cdots & h_{1k} \\ h_{21} & h_{22} & \cdots & h_{2k} \\ \vdots & \vdots & \vdots & \vdots \\ h_{r1} & h_{r2} & \cdots & h_{rk} \end{bmatrix}$$

Mã khối

- ▶ Số lượng bit khác nhau giữa 2 từ mã gọi là khoảng cách Hamming.
- ▶ Với mã sửa được x lỗi thì khoảng cách Hamming giữa các từ mã là $2x + 1$.
- ▶ Ở phía thu, dựa trên dữ liệu nhận được ta sẽ tính lại các bit kiểm tra và so sánh với các bit kiểm tra nhận được; nếu khác nhau thì có ít nhất 1 lỗi sai xảy ra.

Mã khối

► Ta có:

$$c = d \cdot G = d \cdot [I_k \quad P] = [d \quad d \cdot P] = [d \quad c_P]$$

► Nếu nhận đúng thì:

$$\begin{aligned} d \cdot P \oplus c_P &= 0 \\ \Leftrightarrow [d \quad c_P] \begin{bmatrix} P \\ I_{n-k} \end{bmatrix} &= 0 \end{aligned}$$

► Ta gọi ma trận mới kích thước $n \times r$ là $H^T = \begin{bmatrix} P \\ I_{n-k} \end{bmatrix}$

Mã khối

- ▶ Như vậy ta có:

$$c.H^T = 0$$

- ▶ Ta gọi ma trận chuyển vị của H^T là ma trận kiểm tra (**parity-check matrix**) H

$$H = [P^T \quad I_{n-k}]$$

Mã khối

- Như vậy ma trận H sẽ có dạng:

$$H = \begin{bmatrix} h_{11} & h_{12} & \cdots & h_{1k} & 1 & 0 & \cdots & 0 \\ h_{21} & h_{22} & \cdots & h_{2k} & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ h_{r1} & h_{r2} & \cdots & h_{rk} & 0 & 0 & \cdots & 1 \end{bmatrix}$$

Mã khối

- Gọi r là vecto nhận được:

$$r = c \oplus e$$

- Với e là vecto lỗi

- Ta có:

$$r.H^T = (c \oplus e)H^T = e.H^T \equiv s$$

Mã khối

- ▶ Vecto s gọi là syndrome. Vecto s khác 0 là chỉ số cho biết có 1 hoặc nhiều lỗi xảy ra.
- ▶ Tuy nhiên vecto s bằng 0 không đảm bảo rằng không có lỗi nào xảy ra, vì các mã có thể không nhất thiết phải phát hiện tất cả các mẫu lỗi.

Mã khối

► Khó khăn:

- Một số lượng tương đối lớn các bit kiểm tra phải được thêm vào để sửa một số lỗi nhỏ. Do đó, các khối thông tin kích thước lớn sẽ yêu cầu số lượng lớn bit kiểm tra.
- Để truyền thời gian thực, tốc độ truyền dẫn và băng thông truyền dẫn phải tăng lên khi số lượng bit kiểm tra được tăng lên.

Mã vòng: biểu diễn dưới dạng đa thức

- ▶ Giới hạn Hamming cung cấp một phương pháp để xác định kích thước mã cần thiết để cung cấp khả năng sửa lỗi được chỉ định.
- ▶ Nó không cung cấp phương tiện để thực sự xây dựng mã, để xác định các bit kiểm tra được thêm vào.
- ▶ Mã vòng cung cấp cấu trúc cần thiết này.
- ▶ Ngoài ra, mã vòng có thể được mô tả ở dạng đa thức, cho phép chúng được tạo ra bởi thanh ghi dịch.

Mã vòng: biểu diễn dưới dạng đa thức

- ▶ Một mã vòng được định nghĩa là một mã trong đó một từ mã được tạo thành bằng cách dịch vòng của một từ mã khác; phép cộng mod-2 từng bit giữa 2 từ mã bất kỳ cũng tạo ra từ mã khác.
- ▶ Gọi từ mã $c = (c_1, c_2, \dots, c_n)$ thì $(c_2, c_3, \dots, c_n, c_1)$ và $(c_3, c_4, \dots, c_n, c_1, c_2) \dots$ cũng là từ mã.
- ▶ Biểu diễn dưới dạng đa thức:
$$c(x) = c_1x^{n-1} + c_2x^{n-2} + \dots + c_{n-1}x + c_n$$

Mã vòng: biểu diễn dưới dạng đa thức

- ▶ Phép dịch được thực hiện bằng cách:

$$xc(x) \bmod (x^n + 1)$$

- ▶ Đa thức sinh có dạng:

$$g(x) = x^r + g_{r-1}x^{r-1} + \dots + g_1x + 1$$

- ▶ Đa thức sinh được xác định là ước của $x^n + 1$

- ▶ Ví dụ:

Mã vòng: biểu diễn dưới dạng đa thức

► **Ví dụ:** mã vòng $(7, k)$ với $r = 7 - k$

Ta có: $x^7 + 1 = (x + 1)(x^3 + x^2 + 1)(x^3 + x + 1)$

Như vậy mã $(7, 4)$ với $r = 3$ sẽ có 2 đa thức sinh $(x^3 + x^2 + 1)$ và $(x^3 + x + 1)$

Mã vòng: biểu diễn dưới dạng đa thức

- ▶ Từ mã được tạo ra bằng cách:

$$c(x) = a(x)g(x)$$

- ▶ Với $a(x)$ là đa thức bậc $k - 1$
- ▶ Ta thay lần lượt các đa thức $a(x)$ ta sẽ thu được tất cả các từ mã của mã vòng.

Table 7.2 *Generation of codewords, (7, 3) code, from*
 $g(x) = x^4 + x^3 + x^2 + 1$

$a(x)$	$a(x)g(x)$	Codeword
0	0	0000000
1	$x^4 + x^3 + x^2 + 1$	0011101
x	$x^5 + x^4 + x^3 + x$	0111010
$x + 1$	$x^5 + x^2 + x + 1$	0100111
x^2	$x^6 + x^5 + x^4 + x^2$	1110100
$x^2 + 1$	$x^6 + x^5 + x^3 + 1$	1101001
$x^2 + x$	$x^6 + x^3 + x^2 + x$	1001110
$x^2 + x + 1$	$x^6 + x^4 + x + 1$	1010011

Mã vòng:
 biểu diễn
 dưới dạng
 đa thức

Mã vòng: biểu diễn dưới dạng đa thức

- Đa thức của k bit thông tin:

$$d(x) = d_1x^{k-1} + d_2x^{k-2} + \cdots + d_{k-1}x + d_k$$

- Ta có:

$$\frac{x^{n-k}d(x)}{g(x)} = z(x) + \frac{r(x)}{g(x)}$$

$$\rightarrow \frac{x^{n-k}d(x)}{g(x)} + \frac{r(x)}{g(x)} = z(x) \rightarrow x^{n-k}d(x) + r(x) = z(x)g(x)$$

Mã vòng: biểu diễn dưới dạng đa thức

- ▶ Như vậy từ mã được tạo ra bằng cách:

$$c(x) = a(x)g(x) = x^{n-k}d(x) + r(x)$$

- ▶ Với $r(x)$ là số dư của phép chia $x^{n-k}d(x)$ cho $g(x)$

$$r(x) = \text{rem} \frac{x^{n-k}d(x)}{g(x)}$$

Mã vòng: biểu diễn dưới dạng đa thức

► **Ví dụ:** Xác định từ mã của mã vòng (7,3) với đa thức sinh $g(x) = x^4 + x^3 + x^2 + 1$ với

a) $d = (1,1,1)$

b) $d = (1,0,1)$

Mã vòng: biểu diễn dưới dạng đa thức

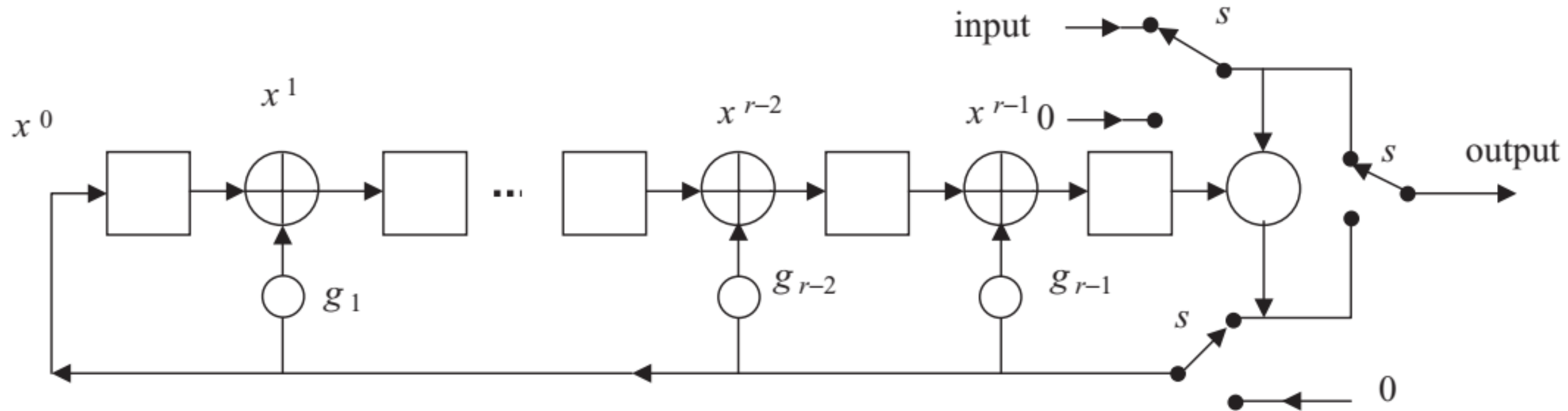


Figure 7.2 Cyclic-code shift-register encoder, r check bits. Switches up for information bits, down for last r parity-check bits

Mã vòng: biểu diễn dưới dạng đa thức

- ▶ Cách thức thực hiện mã vòng bằng thanh ghi dịch ở hình vẽ trên.
- ▶ Các ô vuông đại diện cho độ trễ 1 bit.
- ▶ Các giá trị g_i là các hệ số của đa thức sinh $g(x)$ và có giá trị bằng 0 hoặc 1.
- ▶ k bit dữ liệu được kí hiệu là “input” với khóa đang ở vị trí “up” như hình vẽ.

Mã vòng: biểu diễn dưới dạng đa thức

- ▶ k bit này ngay lập tức được lấy ở đầu ra để tạo thành k bit trong n bit đầu tiên của từ mã.
- ▶ k bit này cũng đồng thời được đi vào các thanh ghi dịch tương ứng và thực hiện các phép toán tương ứng.
- ▶ Sau khi lấy xong k bit thì khóa chuyển xuống vị trí “down”.
- ▶ Lúc này các bit từ thanh ghi dịch sẽ được lấy ra, từng bit một tạo thành r bit kiểm tra.

Mã vòng: biểu diễn dưới dạng đa thức

- ▶ Các thanh ghi dịch và các phép toán vẫn thực hiện cùng lúc trong thời gian này.
- ▶ Các bit 0 được đưa vào để thực hiện các phép toán để xóa các thanh ghi dịch đến khi các bit kiểm tra được lấy ra hết.

Mã vòng: biểu diễn dưới dạng đa thức

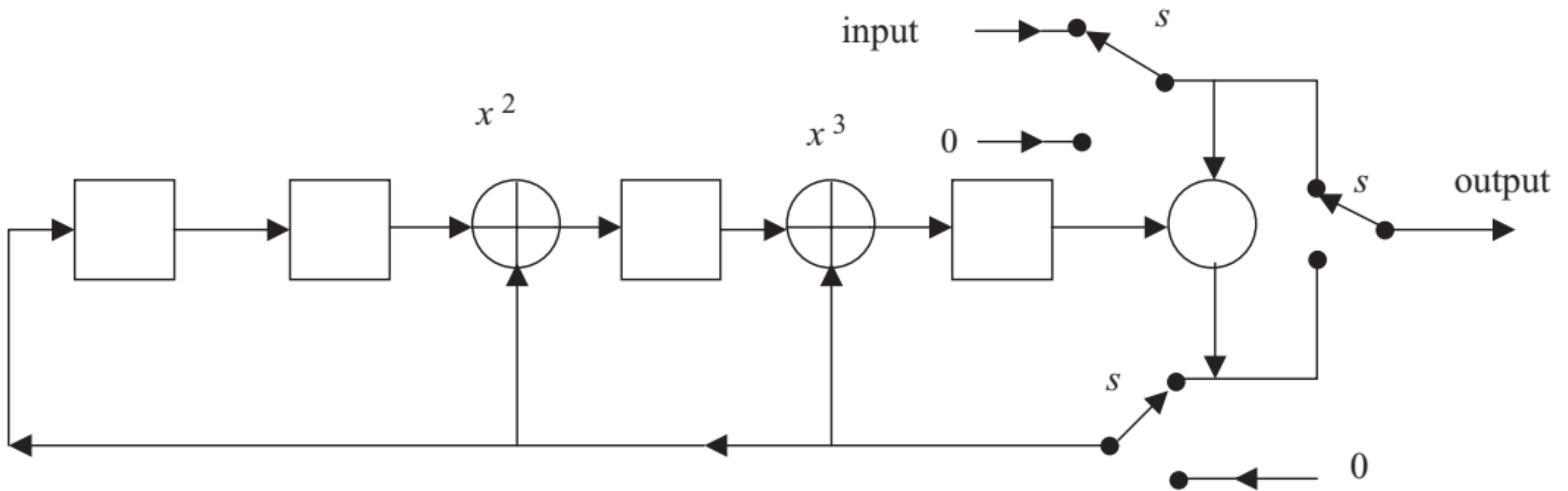


Figure 7.3 Encoder, (7, 3) code, $g(x) = x^4 + x^3 + x^2 + 1$

Mã vòng: biểu diễn dưới dạng đa thức

► **Ví dụ:** vẽ sơ đồ tạo mã vòng (7,4) với đa thức sinh

$$g(x) = x^3 + x^2 + 1$$

Phát hiện lỗi với mã vòng

- ▶ Ở phía phát, các thanh ghi dịch được sử dụng để tạo ra từ mã.
- ▶ Ở phía thu, tương tự, các thanh ghi dịch cũng được dùng để tái tạo lại các bit kiểm tra và so sánh với các bit kiểm tra thực tế nhận được để xem có bất kỳ lỗi nào xảy ra trong dữ liệu nhận được.
- ▶ Mục đích sử dụng chính của mã khối trong các hệ thống di động không dây là để phát hiện, thay vì sửa lỗi.

Phát hiện lỗi với mã vòng

- ▶ Kích thước khối thông tin tương đối lớn được dùng trong các hệ thống không dây kỹ thuật số, có kích thước hàng trăm bit nên không thể sử dụng các kỹ thuật sửa lỗi dựa trên mã khối.
- ▶ Do đó mã kiểm tra được sử dụng để phát hiện lỗi.
- ▶ Các mạng chuyển mạch gói như Internet cũng sử dụng các tính toán kiểm tra chẵn lẻ cho các mục đích phát hiện lỗi.

Phát hiện lỗi với mã vòng

- ▶ Trong trường hợp hệ thống không dây, thông thường sử dụng 8, 12 hoặc 16 bit kiểm tra được gắn vào các khối thông tin để thực hiện phát hiện lỗi.
- ▶ Bộ mã chập và turbo được sử dụng tại máy phát, thực hiện sau quy trình kiểm tra khối phát hiện lỗi, để cung cấp khả năng sửa lỗi trong các hệ thống này.

Phát hiện lỗi với mã vòng

- ▶ Tại máy thu, quy trình ngược lại được thực hiện: bộ giải mã chấp và bộ giải mã turbo trước tiên thực hiện sửa lỗi.
- ▶ Sau đó, quy trình kiểm tra khối được sử dụng để phát hiện bất kỳ lỗi nào không được sửa.
- ▶ Các mã khối sau đó được gọi là mã bên ngoài; mã chấp và mã turbo được gọi là mã bên trong.

Phát hiện lỗi với mã vòng

- ▶ Phát hiện lỗi được xử lý theo nhiều cách trong các hệ thống viễn thông:
 - Một khối thông tin chứa một hoặc nhiều lỗi được phát hiện nhưng chưa được sửa có thể bị loại bỏ.
 - Hệ thống nhận có thể báo hiệu rằng một hoặc nhiều lỗi đã được phát hiện để thực hiện hành động tiếp theo.
 - Hệ thống nhận có thể báo hiệu cho bộ phát truyền lại khối có lỗi.

Phát hiện lỗi với mã vòng

- ▶ Cách xử lý loại bỏ khối có lỗi có thể được sử dụng cho lưu lượng thời gian thực, chẳng hạn như thoại.
- ▶ Hệ thống nhận phát hiện lỗi trong khối dữ liệu thoại có thể chỉ cần loại bỏ khối đó.
- ▶ Tỷ lệ lỗi lên tới 5% có thể được chấp nhận trong giọng nói thời gian thực, tức là những người tham gia cuộc gọi thoại không thấy cuộc trò chuyện bị giảm chất lượng quá nhiều do thỉnh thoảng bị mất mẫu giọng nói.

Phát hiện lỗi với mã vòng

- ▶ Các hệ thống thể hệ thứ ba được thiết kế để xử lý cả lưu lượng dữ liệu chuyển mạch gói.
- ▶ Phần lớn loại lưu lượng này không yêu cầu phân phối theo thời gian thực. Trong trường hợp này, truyền lại gói có thể được sử dụng để sửa bất kỳ lỗi nào được phát hiện.
- ▶ Kỹ thuật ARQ, dành cho yêu cầu lặp lại tự động, thường được sử dụng trong các mạng chuyển mạch gói, bao gồm cả Internet và cũng đã được áp dụng cho các hệ thống không dây chuyển mạch gói.

Phát hiện lỗi với mã vòng

- ▶ Trong kỹ thuật này, một bộ đếm thời gian được đặt tại hệ thống truyền khi gửi một khối dữ liệu.
- ▶ Nếu hệ thống nhận không phát hiện lỗi nào, nó sẽ phản hồi bằng một xác nhận tích cực, hệ thống gửi sẽ nhận được trước khi hết giờ.
- ▶ Mặt khác, nếu phát hiện ra lỗi, bên nhận chỉ cần loại bỏ khối. Sau đó, bộ hẹn giờ tại hệ thống gửi sẽ hết thời gian, với khối dữ liệu sau đó sẽ tự động gửi lại và bộ hẹn giờ được đặt lại.

Phát hiện lỗi với mã vòng

- ▶ Một mã vòng sử dụng r bit kiểm tra sẽ phát hiện một tỷ lệ phần trăm cao các lỗi chum có chiều dài lớn hơn r .
- ▶ Bằng cách sử dụng biểu diễn đa thức của mã vòng, người ta dễ dàng chứng minh rằng tỷ lệ các lỗi chùm có độ dài b lớn hơn r vẫn không bị phát hiện chỉ là 2^{-r}
- ▶ Kết quả này không phụ thuộc vào độ dài của khối thông tin. Do đó, các khối dài có thể được sử dụng và hầu hết các mẫu lỗi sẽ được phát hiện.

Phát hiện lỗi với mã vòng

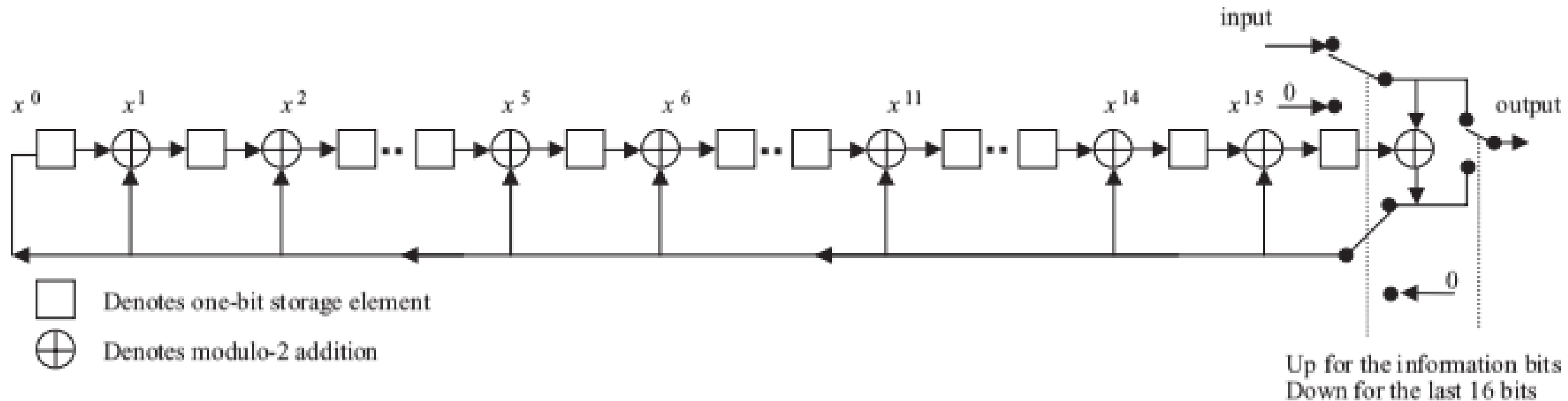


Figure 7.4 Frame quality indicator calculation for the 16-bit frame quality Indicator, cdma2000 (from cdma, 2002: Fig. 2.1.3.1.4.2.1-1)

Phát hiện lỗi với mã vòng

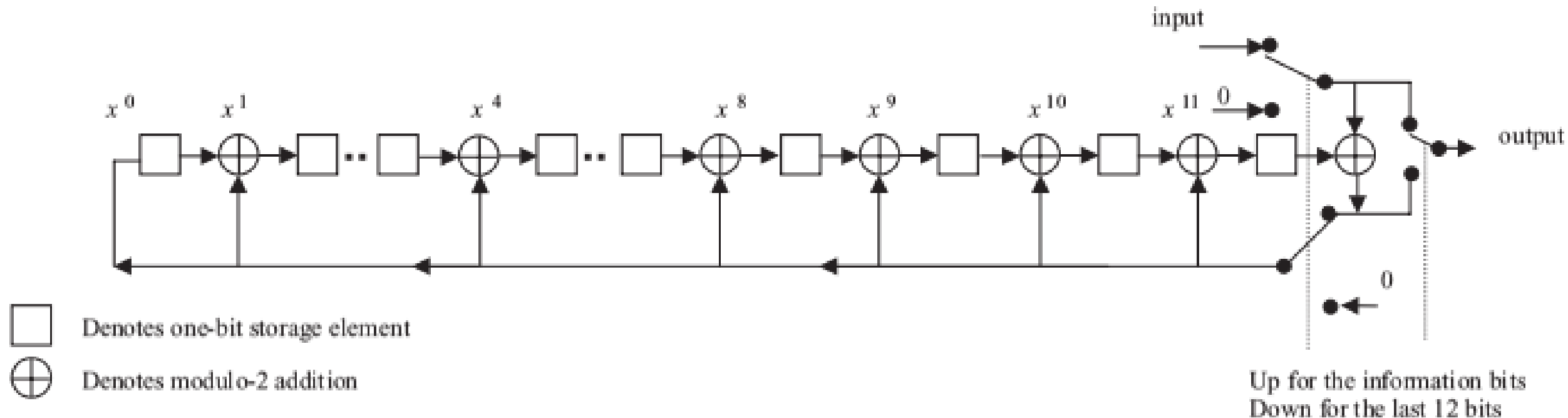


Figure 7.5 Frame quality indicator calculation for the 12-bit frame quality Indicator, cdma2000 (from cdma, 2000: fig. 2.1.3.1.4.2.1-2)

Phát hiện lỗi với mã vòng

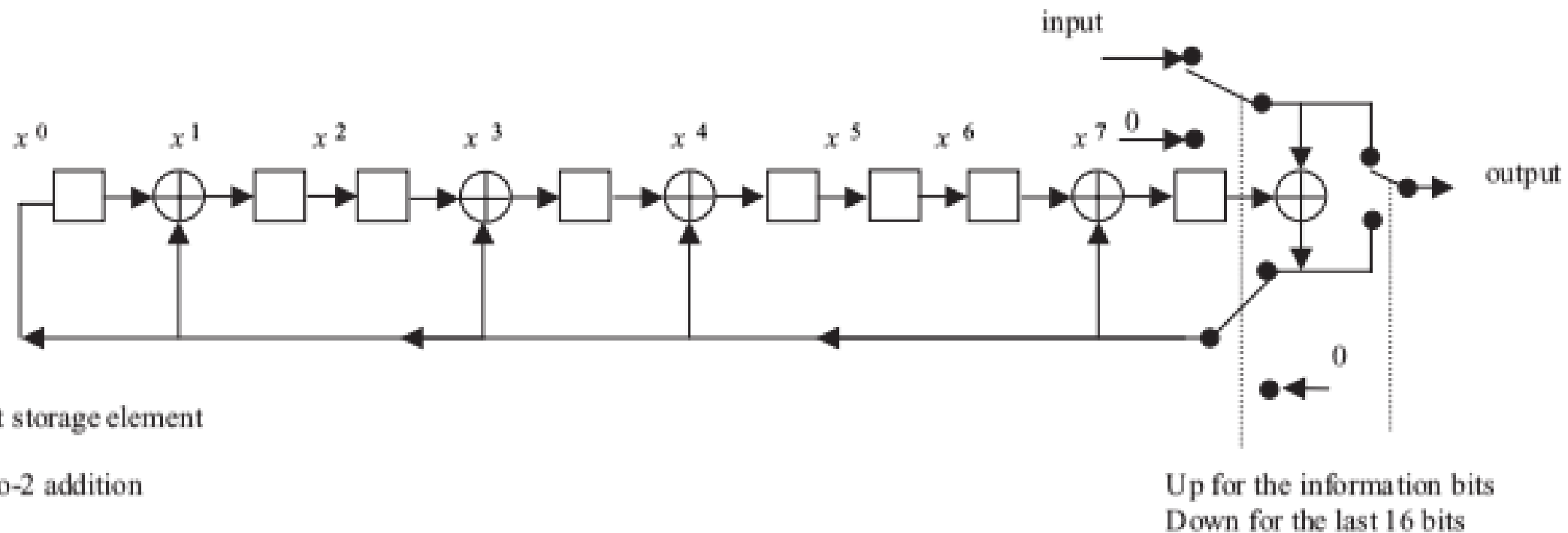


Figure 7.6 Frame quality indicator calculation for the 8-bit frame quality Indicator, cdma2000 (from cdma, 2002: fig. 2.1.3.1.4.2.1-3)

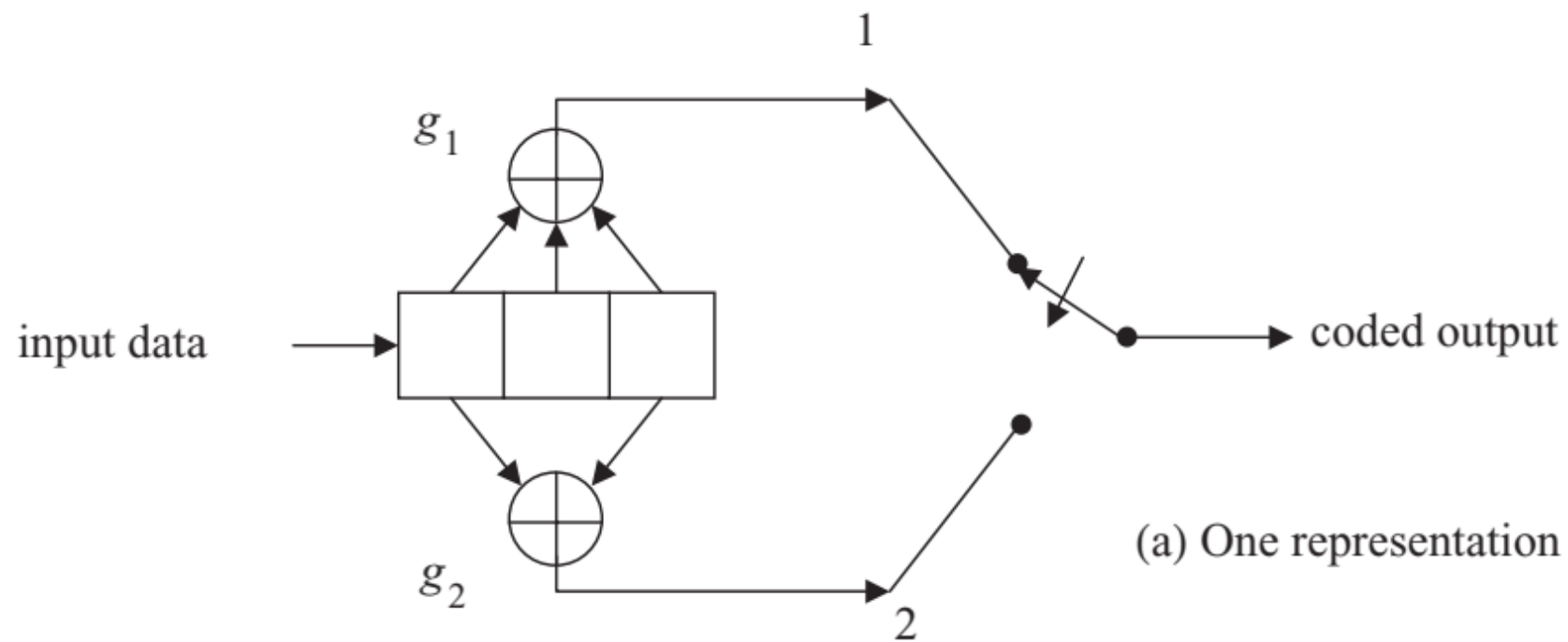
Mã chập

- ▶ Bộ mã chập có khả năng sửa lỗi và lần đầu tiên được giới thiệu để sử dụng trong các ứng dụng liên lạc vệ tinh và không gian.
- ▶ Các hệ thống di động kỹ thuật số hiện nay cũng sử dụng các bộ mã hóa này.
- ▶ Bộ mã hóa tích chập bao gồm một thanh ghi dịch có K tầng.
- ▶ Các bit dữ liệu đầu vào được dịch chuyển dọc theo thanh ghi dịch từng bit một.

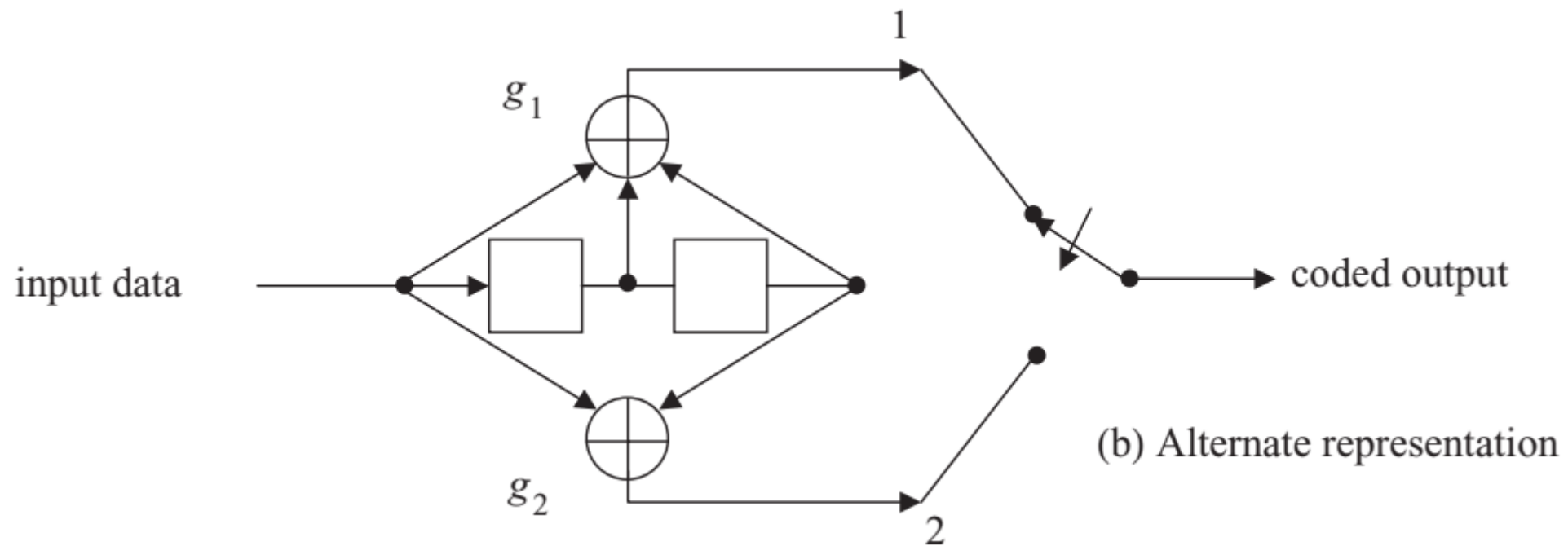
Mã chập

- ▶ Phép cộng modulo-2 sẽ tạo ra các bit đầu ra: 1 bit đầu vào sẽ tạo ra v bit đầu ra.
- ▶ K được gọi là độ dài ràng buộc của bộ mã hóa.
- ▶ Tốc độ mã là $1/v$.

Mã chập



Mã chập



Mã chập

Input bits:	0	1	1	0	1	0	0	1							
Output bits:	0	0	1	1	0	1	0	1	0	0	1	0	1	1	1

- **Ví dụ:** xác định chuỗi bit đầu ra với chuỗi bit đầu vào là 10010011 sử dụng mã chập như hình vẽ trên.

Mã chập

- ▶ Ở hình vẽ bộ mã trên, các kết nối vào bộ modulo-2 có thể được biểu diễn bằng 2 đa thức sinh g_1 và g_2
- ▶ Mỗi đa thức sinh này được đại diện bằng vecto K bit, mỗi bit tương ứng với kết nối với mỗi tầng của thanh ghi dịch (bằng 1 nếu có kết nối, bằng 0 nếu không có kết nối).
- ▶ **Ví dụ:** với hình vẽ trên thì $g_1 = [111]$ và $g_2 = [101]$

Mã chập

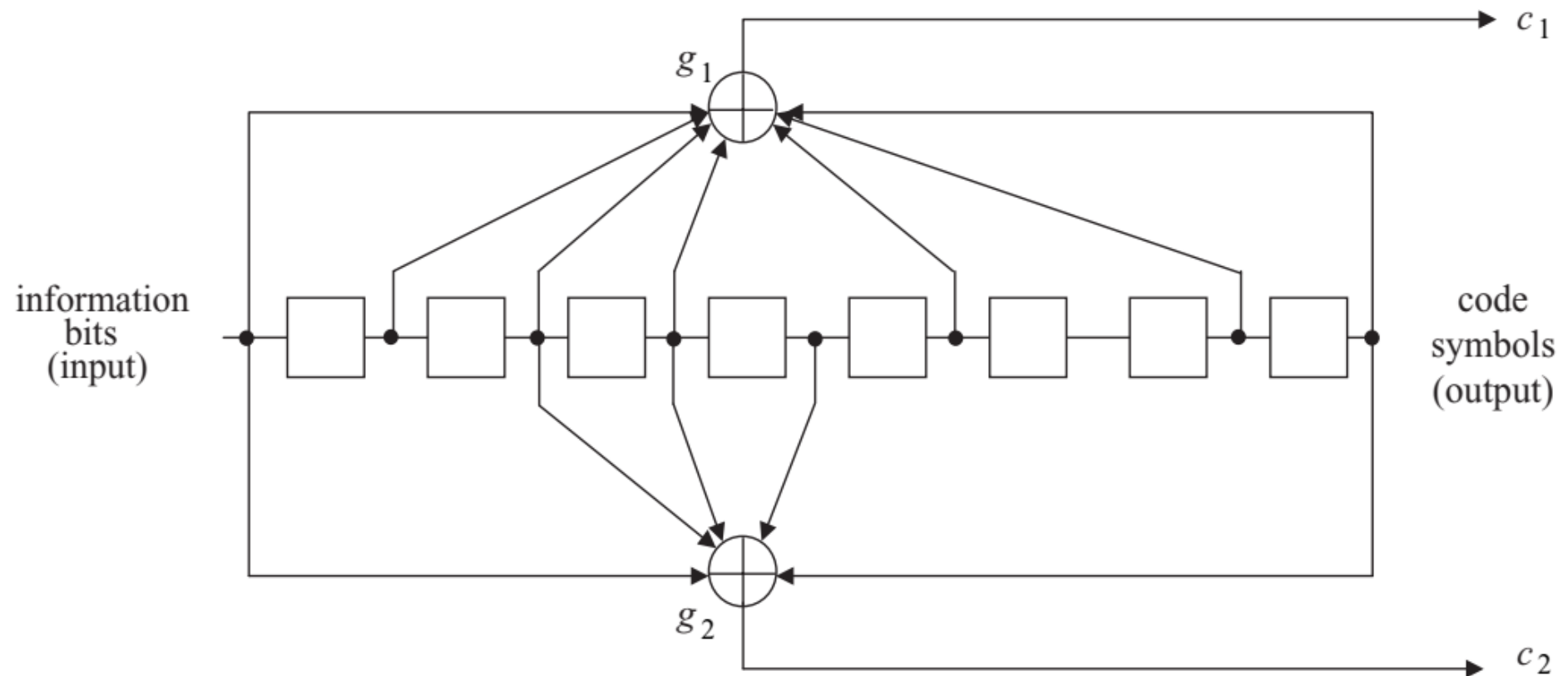


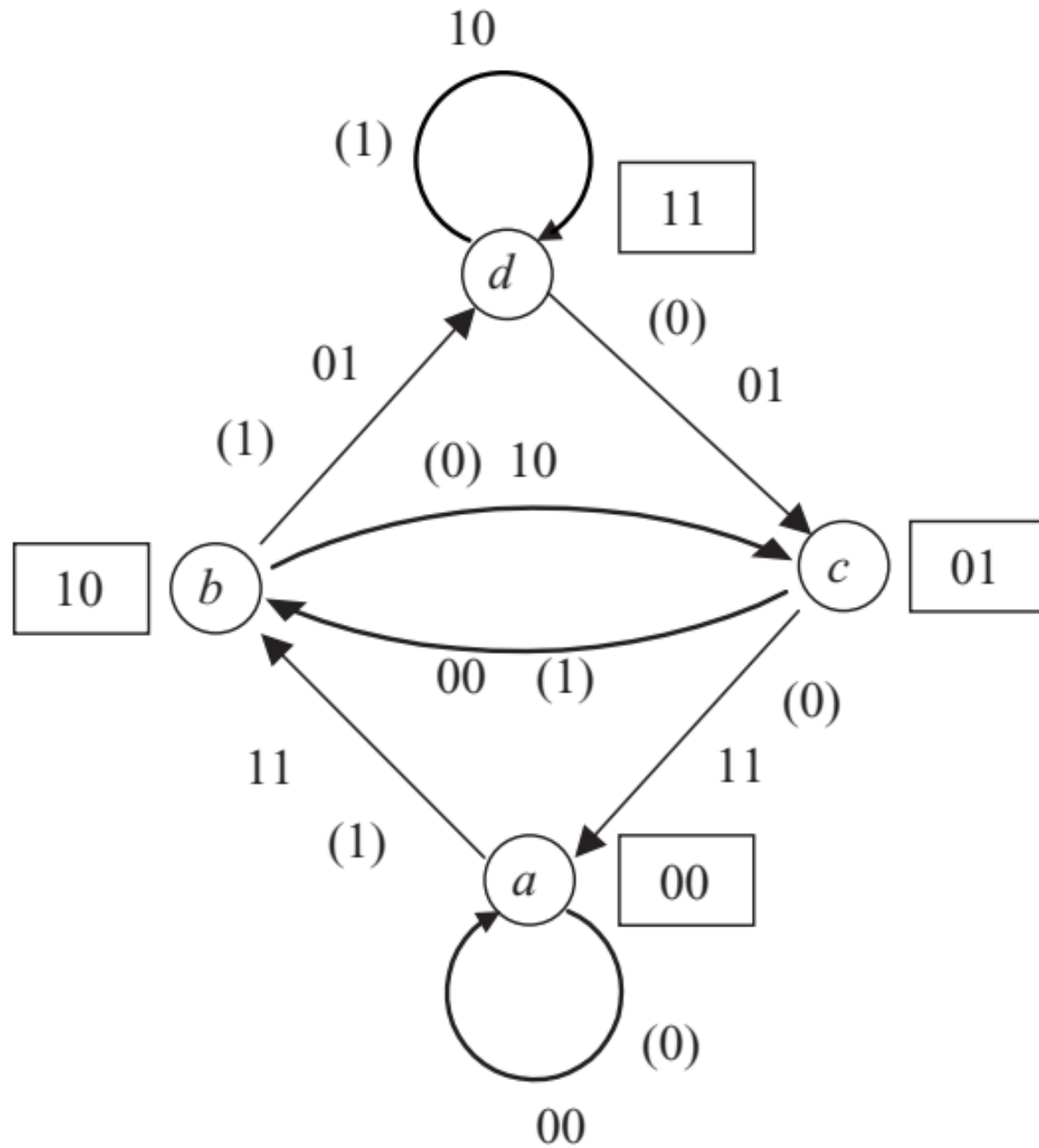
Figure 7.8 $K = 9$, rate $1/2$ convolutional encoder (from cdma, 2002: fig. 2.1.3.1.4.1.3-1)

Mã chập

- ▶ Ví dụ: Xác định đa thức sinh của bộ mã trên.

Mã chập

- ▶ Bộ mã hóa chập có thể được biểu diễn bằng giản đồ trạng thái, giản đồ Trellis hay giản đồ cây.



Mã chập

Figure 7.9 State diagram, convolutional encoder of Fig. 7.7

Mã chập

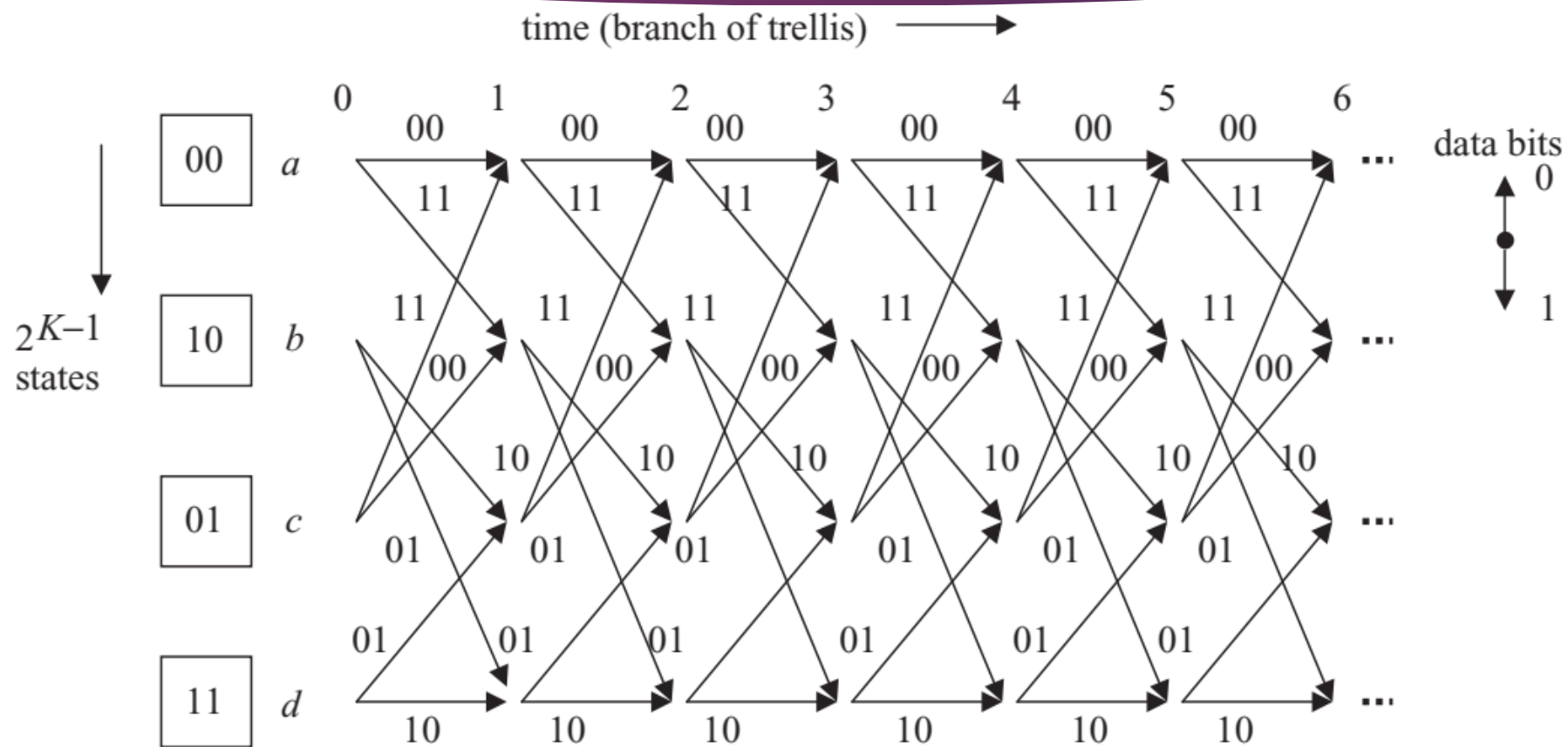


Figure 7.10 Trellis representation, encoder of Fig. 7.7

Mã chập

- ▶ Biểu diễn tín hiệu nhận được bằng nhiều hơn một bit và sử dụng biểu diễn này để đưa ra quyết định được gọi là giải mã quyết định mềm.
- ▶ Thuật toán Viterbi được sử dụng giúp mang lại hiệu suất gần như tối ưu.
- ▶ Chúng ta chọn ra đường mà có khoảng cách Hamming ngắn nhất so với tín hiệu nhận được.

Mã chập

- ▶ Vấn đề đặt ra là khi số bit L tăng lên thì số con đường phải tính toán là 2^L
- ▶ Một thuật toán giải mã **maximum-likelihood** của Andrew J. Viterbi, thường được gọi là thuật toán Viterbi, giúp giảm bớt đáng kể vấn đề này và đã được sử dụng rộng rãi trong truyền thông vệ tinh và không gian, cũng như các ứng dụng không dây.

Mã chập

- ▶ Để đạt được trạng thái hiện tại, nó chỉ có thể ở một trong hai trạng thái có thể có trong khoảng thời gian trước đó.
- ▶ **Ví dụ:** nếu trạng thái hiện tại là b thì trạng thái trước đó chỉ có thể là a hoặc c. Nếu khoảng cách Hamming đi theo đường a-b lớn hơn đi theo đường c-b thì chắc chắn ở các trạng thái phía sau thì con đường c-b vẫn có khoảng cách Hamming ngắn hơn. Như vậy chúng ta có thể loại bỏ con đường a-b

Mã chập

- ▶ Như vậy với L bit thì số con đường phải so sánh chỉ là $2^{K-1}L$ so với 2^L ban đầu.
- ▶ Các nghiên cứu đã chỉ ra rằng giá trị của L khoảng bốn hoặc năm lần K là đủ để tạo ra hiệu suất gần với giá trị của L rất lớn.

Mã chập

Table 7.3 Viterbi algorithm, 5-interval operation, $K = 3$, rate-1/2 coder

interval→ received bits→	Possible paths					Previous state	Current state	Hamming distance
	1	2	3	4	5			
	01	01	00	11	10			
	11	01	01	11	00	a	a	$2 + 1 = 3^*$
	11	01	10	01	11	c	a	$3 + 1 = 4$
	11	01	01	11	11	a	b	$2 + 1 = 3^*$
	11	01	10	01	00	c	b	$3 + 1 = 4$
	00	00	00	11	10	b	c	$2 + 0 = 2^*$
	11	01	10	10	01	d	c	$3 + 2 = 5$
	00	00	00	11	01	b	d	$2 + 2 = 4$
	11	01	10	10	10	d	d	$3 + 0 = 3^*$